

CQRS

Por: Ricardo Cuellar



¿Qué es CQRS?

CQRS (Command Query Responsibility Segregation)

- Separación de responsabilidad entre comandos y consultas.

La idea principal es separar las operaciones que modifican información de las operaciones que solamente leen información.



Sin CQRS

En una aplicación tradicional normalmente tenemos un solo servicio o repositorio con métodos como:

```
example.go

// Sin CQRS - todo en una interfaz, crece sin límite
type TaskRepository interface {
    Create(ctx context.Context, task *domain.Task) error
    Update(ctx context.Context, task *domain.Task) error
    Delete(ctx context.Context, id string) error
    GetByID(ctx context.Context, id string) (*domain.Task, error)
    ListByProject(ctx context.Context, projectID string, p Pagination) ([]*domain.Task, error)
    ListByAssignee(ctx context.Context, userID string) ([]*domain.Task, error)
    SearchByTitle(ctx context.Context, query string) ([]*domain.Task, error)
    GetTasksCompletedThisWeek(ctx context.Context, projectID string) (int, error)
    // ... crece, crece y crece
}
```



Con CQRS

Se separan las operaciones en dos grupos por intención.

Commands: Son acciones que buscan cambiar el estado del sistema.

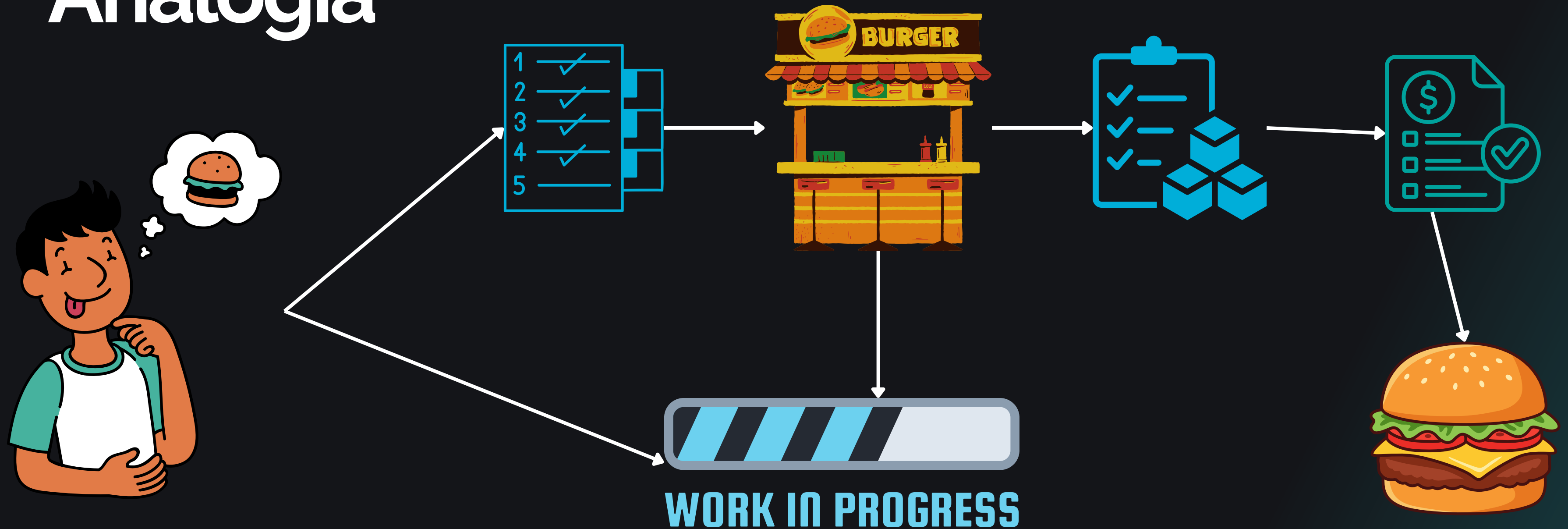
- CreateUser
- UpdateUser
- DeleteUser
- AssignTask
- Complete Task

Queries: Son operaciones que únicamente consultan datos.

- GetUserByID
- ListUser
- GetTaskDetails
- SearchTask
- GetDashboard



Analogía



Con CQRS

```
example.go

// Con CQRS – separado por intención
type TaskCommandRepository interface {
    Create(ctx context.Context, task *domain.Task) error
    Update(ctx context.Context, task *domain.Task) error
    Delete(ctx context.Context, id string) error
}

type TaskQueryRepository interface {
    GetByID(ctx context.Context, id string) (*domain.Task, error)
    ListByProject(ctx context.Context, projectID string, p Pagination) ([]*domain.Task, error)
    SearchByTitle(ctx context.Context, query string) ([]*domain.Task, error)
}
```



¿Por qué separarlos?

Por las necesidades de las lecturas y escrituras, así como lo que les importa.

Escritura:

- Validar reglas de negocio
- Mantener consistencia
- Ejecutar transacciones
- Verificar permisos
- Evitar duplicados
- Publicar eventos

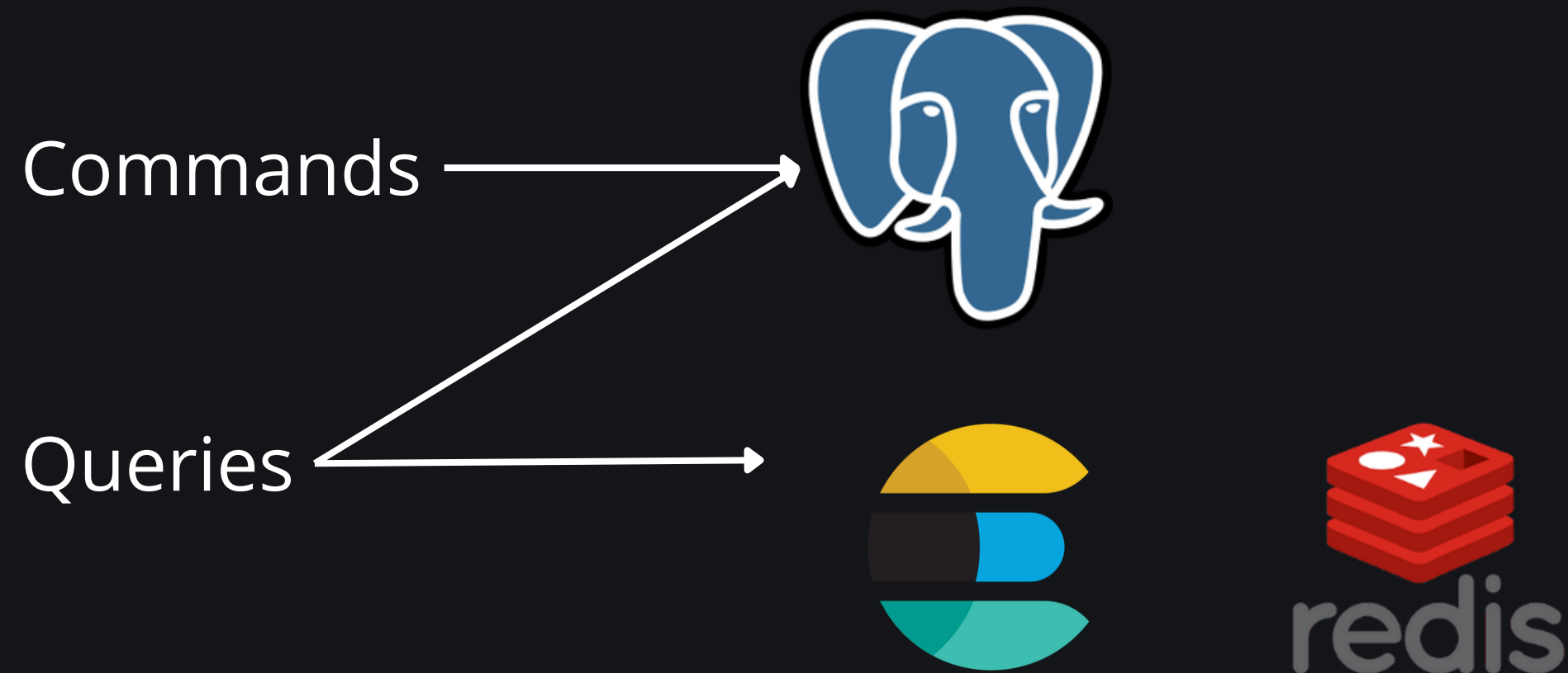
Lectura:

- Consultas rápidas
- Filtrar
- Ordenar
- Paginar
- Combinar tablas
- Devolver datos preparados



CQRS no es igual a dos bases de datos

Se puede aplicar CQRS usando la misma base de datos, solo usando handlers separados.



CQRS simple

Es cuando separamos interfaces o casos de uso:

CreateTask

UpdateTask

DeleteTask

GetTask

ListTask

SearchTask

Conservar la misma base de datos. Suficiente para aplicaciones backend medianas.



CQRS avanzado

Se utilizan:

- Modelos de escritura distintos
- Modelos de lectura distintos
- Bases de datos separadas
- Eventos
- Colas
- Proyecciones
- Consistencia eventual



¿Cuándo usarlo?

- Las lecturas son mucho más frecuentes que las escrituras
- Las consultas son complejas
- El modelo de lectura es diferente al modelo de dominio
- Existen muchas reglas de negocio
- Quieres casos muy explícitos
- Necesitamos escalar las lecturas y escrituras
- El sistema maneja eventos o procesos asíncronos



¿Cuándo NO usarlo?

- La aplicación es pequeña.
- Solamente tienes CRUD.
- Cuando no se domina la arquitectura actual del proyecto.
- Las consultas y escrituras son sencillas.
- La separación agregaría más archivos que valor.
- No existen problemas reales de escalabilidad.



Gracias



www.devtalles.com



[/ricardocuellars](https://www.linkedin.com/company/ricardocuellars)



[@ricardocuellars](https://twitter.com/ricardocuellars)

